Data, Computation & Innovation II

Instructor: Jared Whalen

Housekeeping

- Course GitHub page
- Video walkthroughs

Course Overview

Week 1 (June 3): Chart Fundamentals + SVG/Svelte Introduction

Week 2 (June 10): D3 Fundamentals, Part 1

Week 3 (June 17): D3 Fundamentals, Part 2

Week 4 (June 24): Interactivity and animation *+ Guest speaker Kavya Beheraj/The Athletic*

Week 5 (July 1): GIS and Mapping *+ Guest speaker Alvin Chang/The Pudding*

Week 6 (July 8): Scrollytelling + Advanced patterns

Week 7 (July 15): Final project presentations

✨ Review ✨

- What are the two main settings of a D3 scale and what does each do?

- In Svelte, how do we pass values from the parent to the child component (top down)? Conversely, how do we pass values from the child to the parent component (bottom up)?

- You have an array of 50 cities and a `$state` variable called `selectedRegion`. You want a filtered version of the data that updates automatically whenever `selectedRegion` changes. How would you define that filtered dataset?

Critique (~10 minutes)

Break into groups of ~3 and discuss last week's assignment

- **Author:** Explain what you did and why
- **Editor:** Give specific questions, give specific feedback
  - what's working?
  - what's unclear?
- **Everyone:** What edits would resolve these tensions?

D3 Fundamentals, Part 1

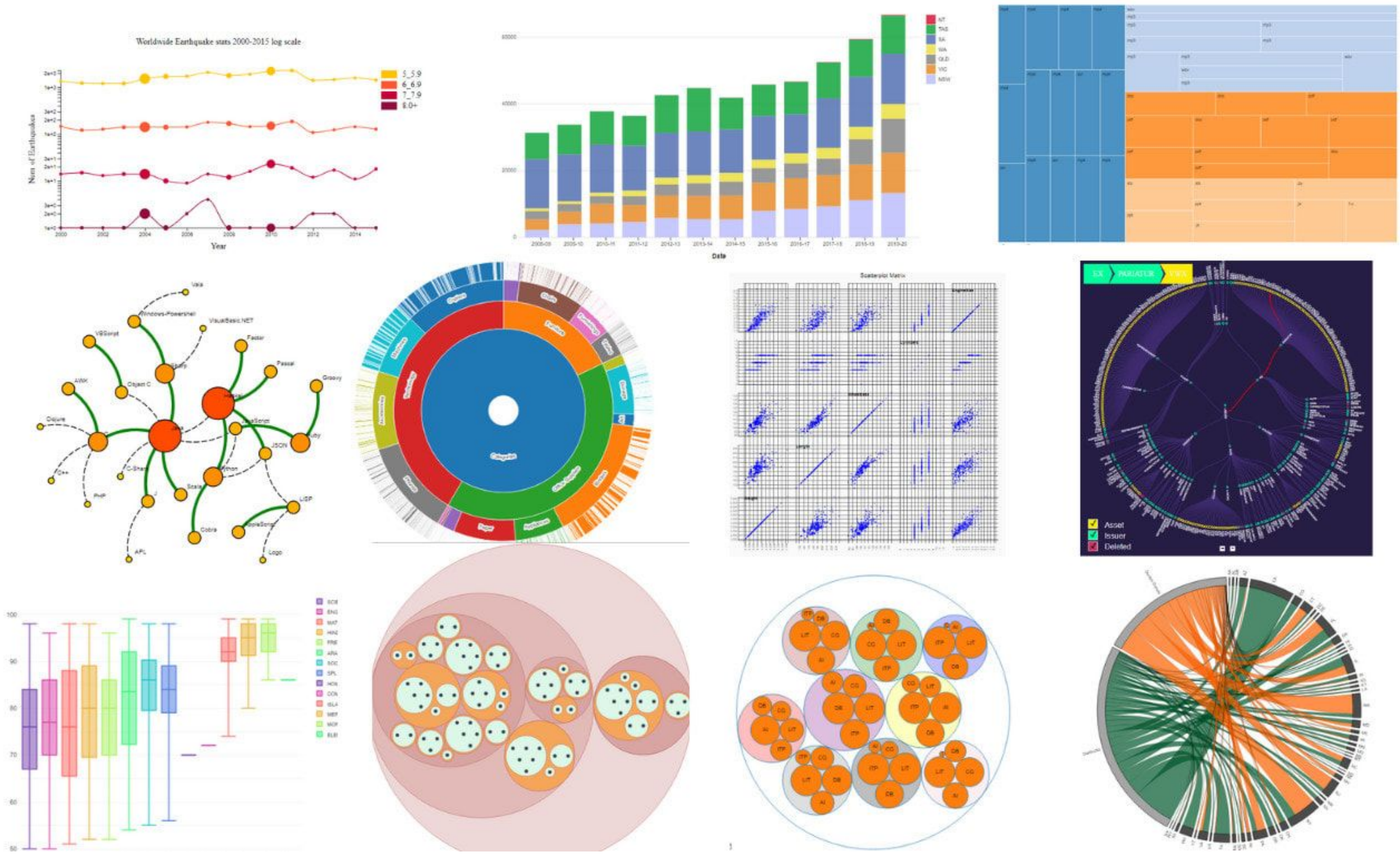
Generators

What is a generator?

Last segment, D3 gave us functions that translate data values into pixels. Shape generators do something similar — they take your entire data array and return a single SVG path string.

You've already written paths by hand using `<polyline>`. This is the same idea, but D3 handles the math.

```
const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))

lineGenerator(data)  // "M 60,278 L 160,265 L 260,232..."
```

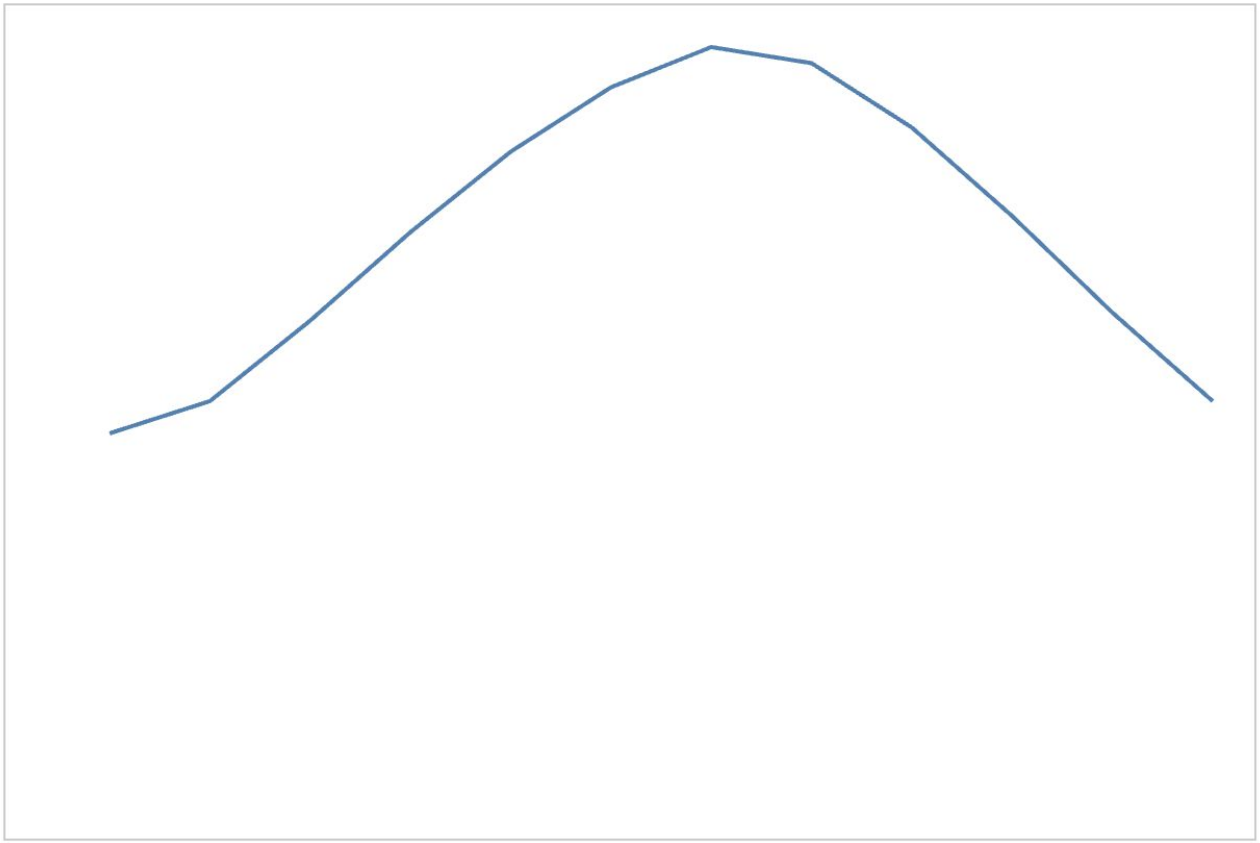

Worldwide Earthquake stats 2000-2015 log scale
Num of Earthquakes
Year
5_5.9
6_6.9
7_7.9
8.0+
Date
Scatterplot Matrix
Vala
Windows-Powershell
VisualBasic.NET
VBScript
Factor
Pascal
Groovy
AWK
Object C
Clojure
JavaScript
JSON
Ruby
C-Sharp
C++
PHP
Python
Scala
LISP
Cobra
AppleScript
APL
Logo
Categories
Asset
Issuer
Deleted
ITP
DB
LIT
CG
AI

d3.line()

Takes an array of data points and returns a path string connecting them.

`.x()` and `.y()` are accessor functions. They receive each data point and return the pixel position for that axis — the same logic you've been writing as inline attributes in `{#each}` loops.

```
const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
</script>

<path
  d={lineGenerator(data)}
  fill="none"
  stroke="steelblue"
  stroke-width="2"
/>
```



d3.area()

Works the same way as `d3.line()` but produces a filled shape between two lines.

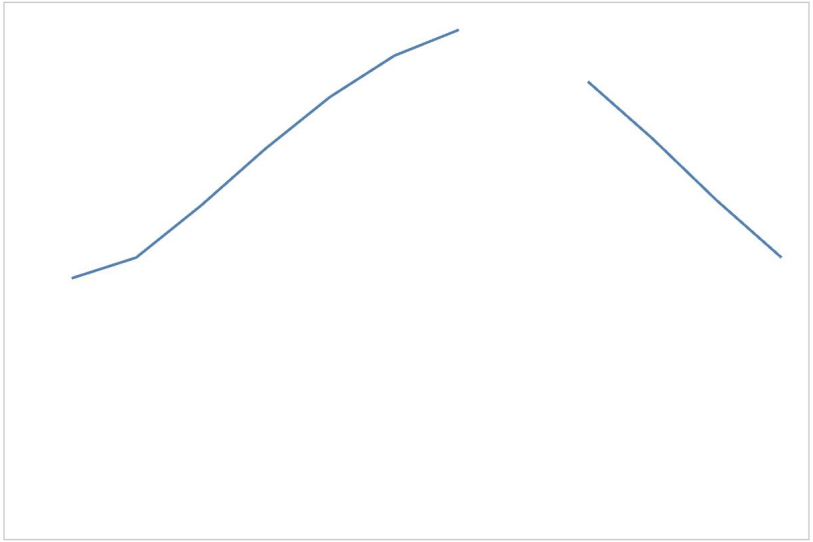
`.y0()` is the baseline — usually the bottom of the chart. `.y1()` is the data line. Everything between them gets filled.

```
const areaGenerator = d3.area()
  .x(d => xScale(d.month))
  .y0(height - margin.bottom)  // baseline
  .y1(d => yScale(d.temp))     // data line
```

Handling missing data

Real datasets often have gaps. By default, `d3.line()` will draw through missing values. You can tell the generator to break the line instead, gracefully handling null values, using the `defined()` method.

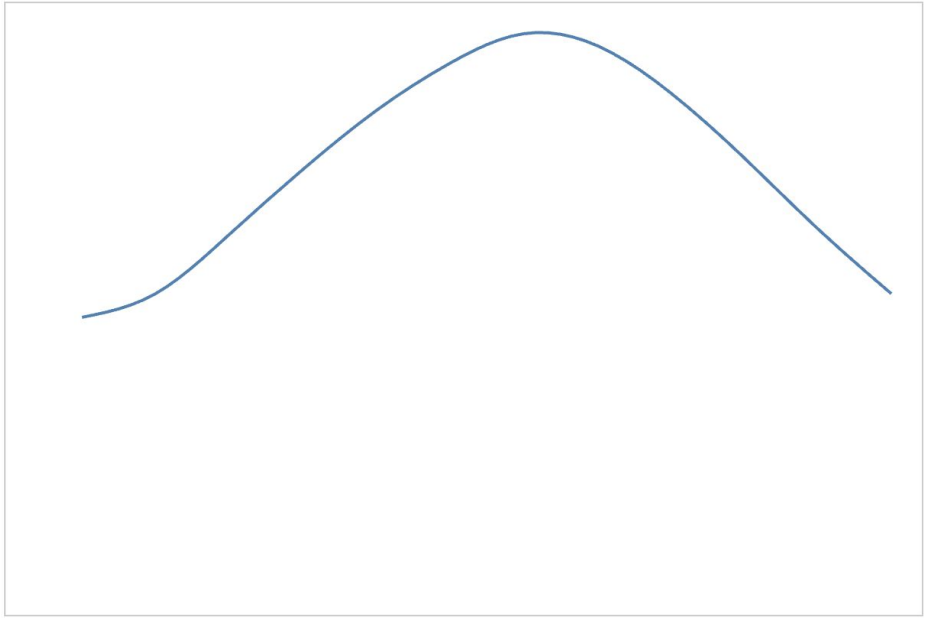
```
const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
  .defined(d => d.temp !== null)
```

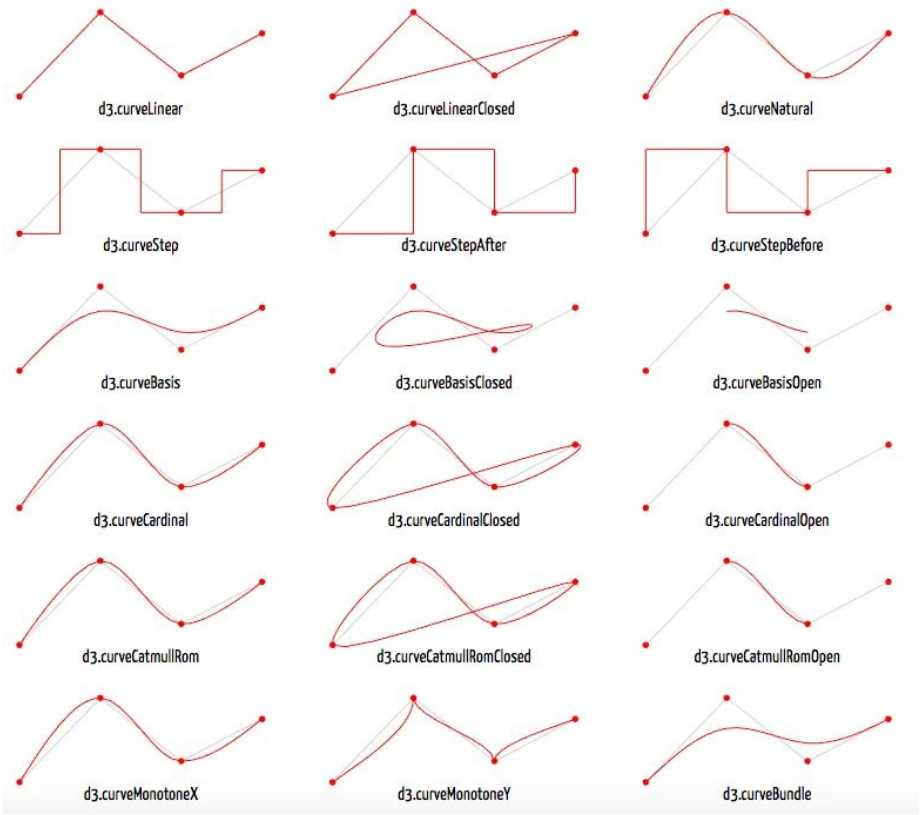


Curves

By default `d3.line()` draws straight segments between points. You can change this with the `.curve()method` to access one of D3's many built in curve functions

```
const lineGenerator = d3.line()
  .x(d => xScale(d.month))
  .y(d => yScale(d.temp))
  .curve(d3.curveNatural)
```





Source: [Learn D3.js]

Axes: the concept

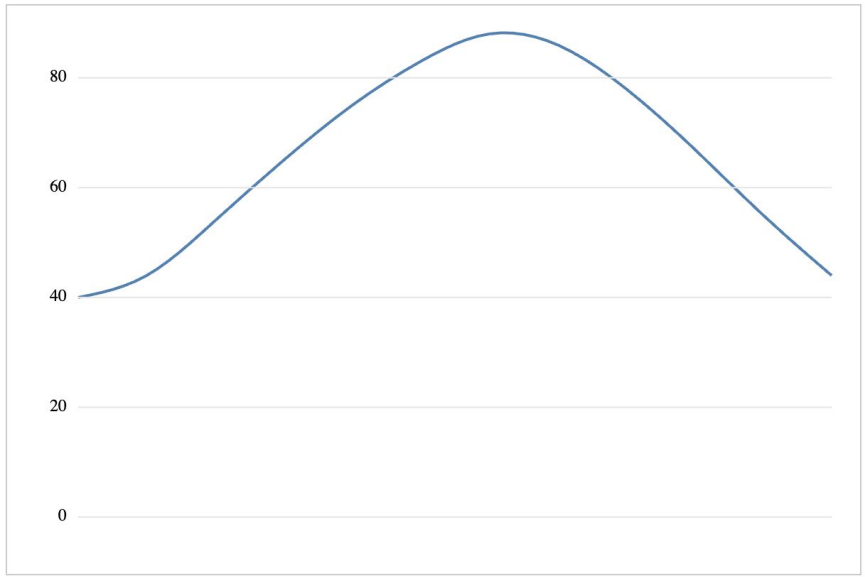
Axes follow the same pattern as line and area generators. D3 gives you the data — tick values and their pixel positions — and you decide how to render it.

We can generate these tick values using our existing scales and the `.ticks()` method.

`scale.ticks(n)` asks the scale for approximately `n` evenly-spaced values within the domain. The scale picks round numbers automatically.

```
const yScale = d3.scaleLinear()
  .domain([0, d3.max(data, d => d.temp)])
  .range([height - margin.bottom, margin.top])

yScale.ticks(5)  // [0, 20, 40, 60, 80]
```

```
    <!-- y axis gridlines and labels -->
    {#each yScale.ticks(5) as tick}
        <line
            x1={margin.left}
            x2={width - margin.right}
            y1={yScale(tick)}
            y2={yScale(tick)}
            stroke="#e5e5e5"
        />
        <text
            x={margin.left - 8}
            y={yScale(tick)}
            text-anchor="end"
            dominant-baseline="middle"
            font-size="12">{tick}</text
        >
    {/each}
</svg>
```



Axes: svg text formatting

`text-anchor` controls horizontal alignment relative to the element's x position:

- "start" — text begins at x
- "middle" — text is centered on x
- "end" — text ends at x

`dominant-baseline` controls vertical alignment relative to the element's y position:

- "auto" — default, baseline sits on y
- "middle" — text is centered on y
- "hanging" — top of text sits on y

```
<!-- centered label -->
<text
    x={xScale(d.month)}
    y={height - 10}
    text-anchor="middle"
>
    {d.month}
</text>

<!-- right-aligned label -->
<text
    x={margin.left - 8}
    y={yScale(tick)}
    text-anchor="end"
    dominant-baseline="middle"
>
    {tick}
</text>
```

Putting it all together

Remember that order matters in SVG — elements render in document order, so later elements appear on top of earlier ones.

```
<script>
  import * as d3 from 'd3'

  // 1. data
  const data = [...]

  // 2. dimensions
  const width = 600
  const height = 400
  const margin = { top: 20, right: 20, bottom: 40, left: 50 }

  // 3. scales
  const xScale = d3.scalePoint().domain(...).range(...)
  const yScale = d3.scaleLinear().domain(...).range(...)

  // 4. generators
  const lineGenerator = d3.line().x(...).y(...)
  const areaGenerator = d3.area().x(...).y0(...).y1(...)
</script>

<!-- 5. rendering -->
<svg viewBox="0 0 {width} {height}" {width} {height}>
  <!-- area -->
  <!-- line -->
  <!-- dots -->
  <!-- axes -->
</svg>
```

 Build Period 

D3 Fundamentals, Part 1

Declarative vs. Imperative

Declarative vs. Imperative

Everything you've built so far has been declarative. You describe what the output should look like and Svelte renders it. The SVG structure is visible in your markup — you can read it without running the code.

Most D3 examples you'll find in the wild are written differently. D3 selects DOM elements and builds the chart directly, step by step. The structure only exists after the JavaScript runs.

These two approaches are called declarative and imperative.

Declarative — describe what you want.

Imperative — describe what to do

Declarative: describe what you want

The structure is in the markup. You can see the `<path>` and the `<circle>` elements before anything runs.

```
<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
  {#each data as d}
    <circle cx={xScale(d.month)} cy={yScale(d.temp)} r="4" fill="steelblue" />
  {/each}
</svg>
```

Imperative: describe what to do

D3 builds the DOM directly. No markup to read — the structure is constructed at runtime.

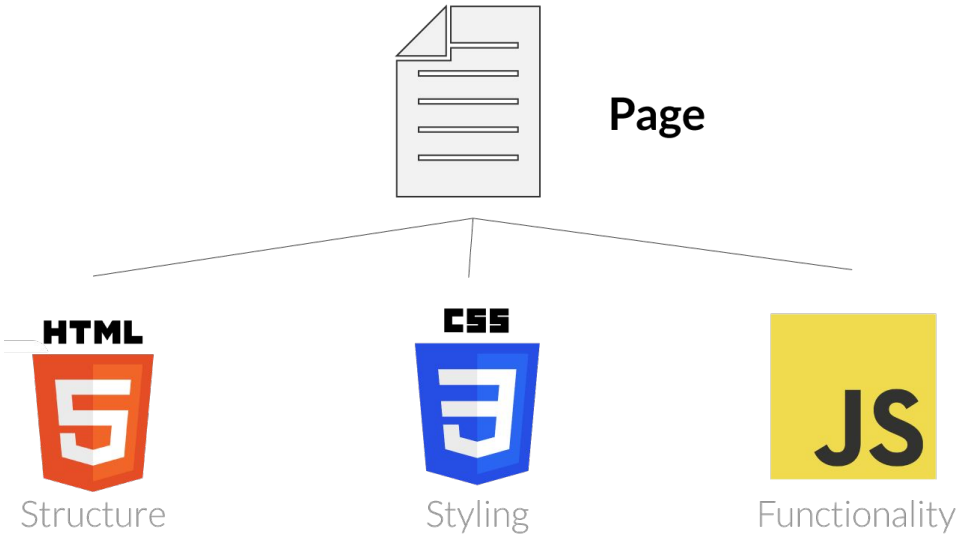
```
const svg = d3.select("#chart")
  .append("svg")
  .attr("viewBox", `0 0 ${width} ${height}`)
  .attr("width", width)
  .attr("height", height)

svg.append("path")
  .datum(data)
  .attr("d", lineGenerator)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2)

svg.selectAll("circle")
  .data(data)
  .join("circle")
  .attr("cx", d => xScale(d.month))
  .attr("cy", d => yScale(d.temp))
  .attr("r", 4)
  .attr("fill", "steelblue")
```

If most D3 examples are imperative, why aren't we writing that way?

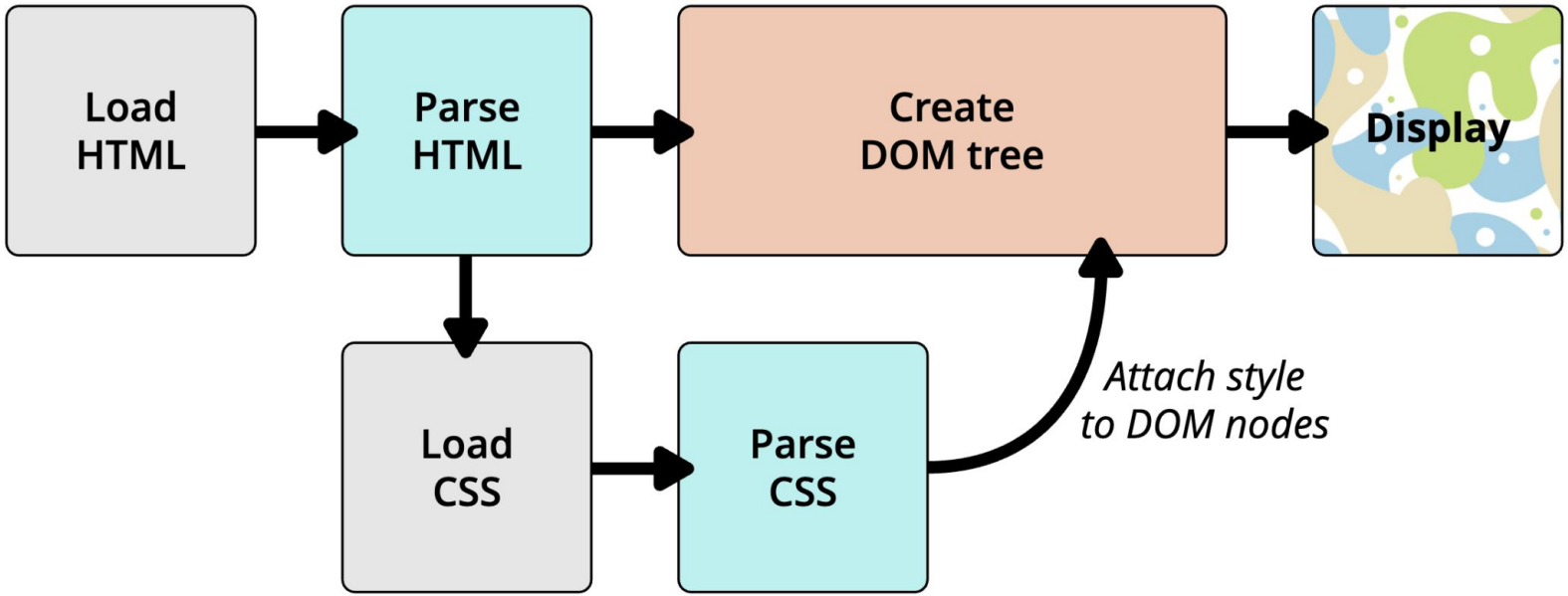
How D3 was originally used

In a plain HTML and JavaScript setup, there's no component framework. You have a static HTML file, a `<script>` tag, and a div with an id. D3 selects that div, appends an SVG, and builds the whole chart imperatively.

That's what the examples you'll find on Observable and GitHub were written for. There's nothing wrong with that approach in that context, it's just a different environment from what we're working in.



If most D3 examples are imperative, why aren't we writing that way?



Source: Mozilla

If most D3 examples are imperative, why aren't we writing that way?

Document Object Model (DOM) timing

Imperative D3 needs to select and write into DOM elements directly. In Svelte, the DOM doesn't exist until the component renders, so any imperative D3 code has to be wrapped in onMount(), which runs after the first render:

In the declarative approach, Svelte handles the rendering — you never need to wait for the DOM because you're not touching it directly.

```
import { onMount } from 'svelte'

onMount(() => {
  // DOM exists here — safe to select el
  d3.select(".my-chart").append("path")
})
```

If most D3 examples are imperative, why aren't we writing that way?

Readability

In the declarative version, the structure of your chart lives in the markup. You can look at the template and see exactly what elements will be on the page (a `<path>`, a set of `<circle>` elements) before anything runs. In the imperative version, the structure is constructed at runtime. You have to run the code mentally to understand what it produces.

Reactivity

Svelte's reactivity (`$state, $derived`) works naturally with the declarative approach. When your data changes, Svelte re-renders the template automatically. With imperative D3, you'd have to manually re-run your D3 code whenever the data changes, and manage the difference between creating elements for the first time and updating existing ones.

Declarative

```
<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
</svg>
```

Imperative

```
const svg = d3.select("#chart")
  .append("svg")
  .attr("width", width)
  .attr("height", height)

svg.append("path")
  .datum(data)
  .attr("d", lineGenerator)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2)
```

The scales and generators are identical

Note, the scale and generator definitions are the same. Nothing about D3's math changes between approaches.

The difference is what you do with the output.

In the **declarative** version, you call the generator and drop the result into a template attribute.

In the **imperative** version, you pass the generator to D3 and let it call it for you.

Same generator. Same output. Different plumbing.

```
<svg viewBox="0 0 {width} {height}" {width} {height}>
  <path
    d={lineGenerator(data)}
    fill="none"
    stroke="steelblue"
    stroke-width="2"
  />
</svg>
```

```
svg.append("path")
  .datum(data)
  .attr("d", lineGenerator)
  .attr("fill", "none")
  .attr("stroke", "steelblue")
  .attr("stroke-width", 2)
```

Reading imperative D3

Since most examples you find on Observable, GitHub, and Stack Overflow are imperative, you need to be able to read them.

A familiar patterns to recognize:

`selectAll` + `data` + `join` is the D3 data join. You can think of this as the imperative equivalent of `{#each}` in that the bound data is iterated over to create one element per data point.

```
d3.select("#chart")        // find an element in the DOM
  .append("svg")           // create a child element
  .attr("width", width)    // set an attribute

svg.selectAll("circle")    // select all circles (even if none exist yet)
  .data(data)              // bind data to the selection
  .join("circle")          // create one element per data point
  .attr("cx", d => ...)    // set attributes using accessor functions
```

Translating imperative to declarative

When you find an example you want to adapt, the process is:

1. **Find the scales:** copy them directly, they're identical in both approaches
2. **Find the generators:** same, copy directly
3. **Find the `selectAll` + `data` + `join` blocks:** these become `{#each}` loops
4. **Find the `.attr()` calls:** these become inline SVG attributes
5. **Find any `d3.select().append()`:** these become explicit SVG elements in your template

There are arguments for both, but a lot of it is preference

	Declarative	Imperative
Readability	Easier to follow	Harder to scan
Svelte reactivity	Works naturally	Requires workarounds
Code in the wild	Less common	Most D3 examples
Complex transitions	Trickier	D3 handles well
Customization	Full control	Fighting D3's output

 Build Period